Search System

為什麼搜尋是一個獨立的設計問題

「搜尋」看起來很簡單：用戶輸入關鍵字，系統回傳相關結果。但如果你用資料庫的 `LIKE` 查詢來實作：

```
SELECT * FROM products WHERE name LIKE '%running shoes%';
```

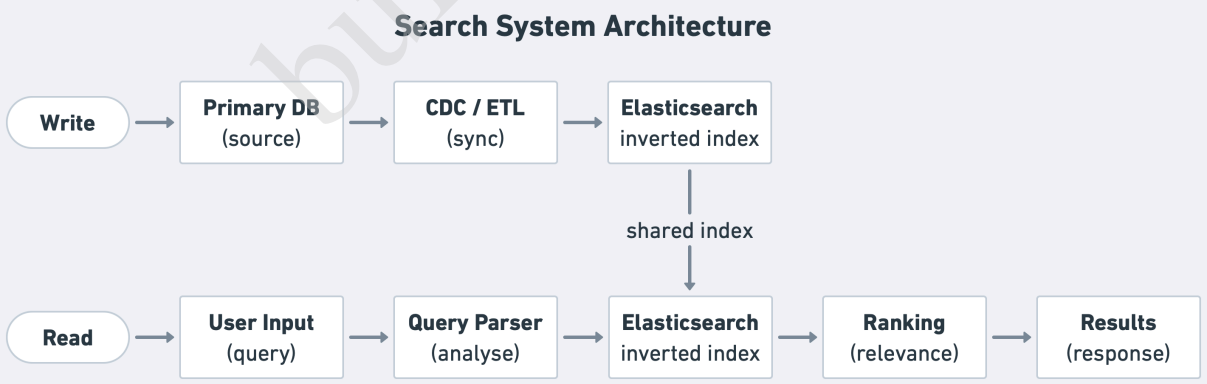
問題立刻出現：這個查詢無法使用索引，會做全表掃描。幾百萬筆商品的時候，這個查詢可能要跑幾十秒。更糟的是，`LIKE '%keyword%'` 只做字串比對，「跑步鞋」不會匹配「慢跑鞋」，「running shoes」不會匹配「run shoe」。

搜尋的本質和資料庫查詢不同。資料庫查詢做的是**精確比對**——找到 user_id = 123 的訂單。搜尋做的是**相關性排序，比如**找出所有和「跑步鞋」相關的商品，按照相關程度排列。這需要完全不同的資料結構和查詢引擎。

這篇講義拆解搜尋系統的三個核心問題：**資料怎麼進入搜尋索引**、**查詢如何運作**、**常見的進階功能怎麼設計**。

搜尋系統的整體架構

一個搜尋系統由兩條獨立的管線組成：



搜尋索引不是 primary database——它是一個**次要索引（secondary index）**，建立在主資料庫上。主資料庫負責資料的寫入和一致性，搜尋索引負責高效的全文搜尋。

倒排索引：搜尋引擎的核心

在設計搜尋系統之前，理解倒排索引（Inverted Index）的基本原理很重要。

buildmoat.org

傳統的資料庫是「文件 → 詞彙」的映射：給我一篇文件，我告訴你它包含哪些詞。倒排索引反過來，建立「詞彙 → 文件」的映射：

Inverted Index term → document list

Product Reference
Nike Running Shoes(P1), Nike Casual Shoes(P3), Adidas Running Shoes(P5), Nike Sport Shoes(P8), Treadmill(P12)

Term	Posting List
“run”	[Product1, Product5, Product12]
“shoe”	[Product1, Product3, Product5, Product8]
“casual”	[Product1, Product5]
“nike”	[Product3, Product8]

Search "run" → [P1, P5, P12] ∩ [P1, P3, P5, P8] =
Nike Running Shoes(P1), Adidas Running Shoes(P5)

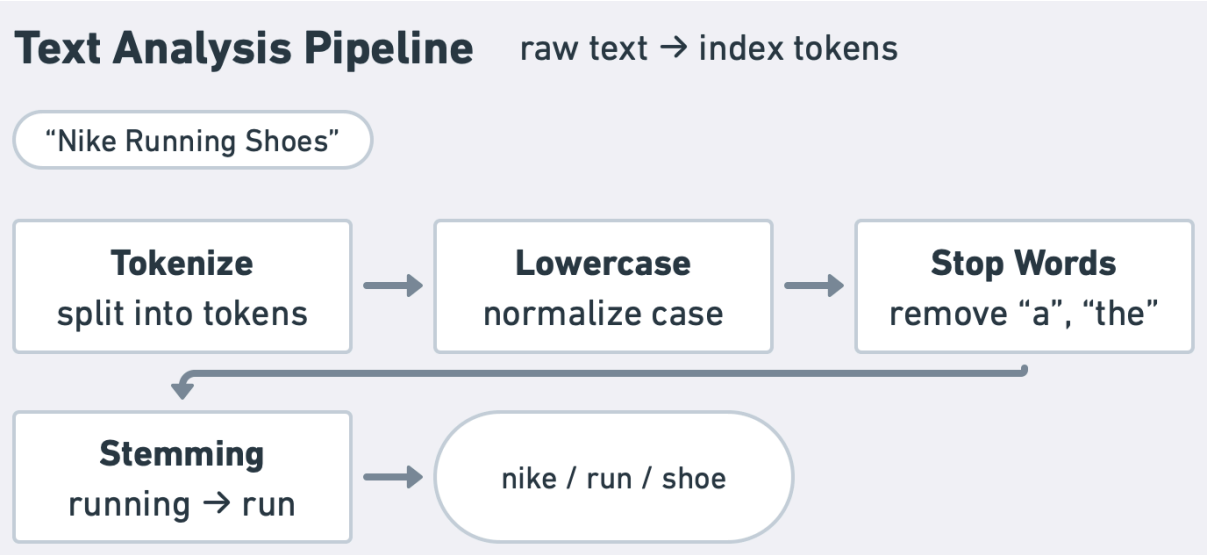
buildmoat.org

搜尋「跑步鞋」時，找出「跑步」和「鞋」各自對應的文件清單，取交集（或聯集），就能快速定位相關文件，而這個操作的速度和全表掃描完全不在一個量級。

文字分析（Text Analysis）：在建立倒排索引之前，原始文字要經過分析處理：

- **Tokenization（斷詞）**：把「Nike Running Shoes」切成 `["nike", "running", "shoes"]`
- **Lowercasing**：統一小寫
- **Stop word removal**：移除「的」、「了」、「a」、「the」等無意義的詞
- **Stemming / Lemmatization**：把「running」、「runs」、「ran」都還原成「run」

查詢時，用戶輸入也經過同樣的分析，確保查詢詞彙和索引詞彙能夠匹配。



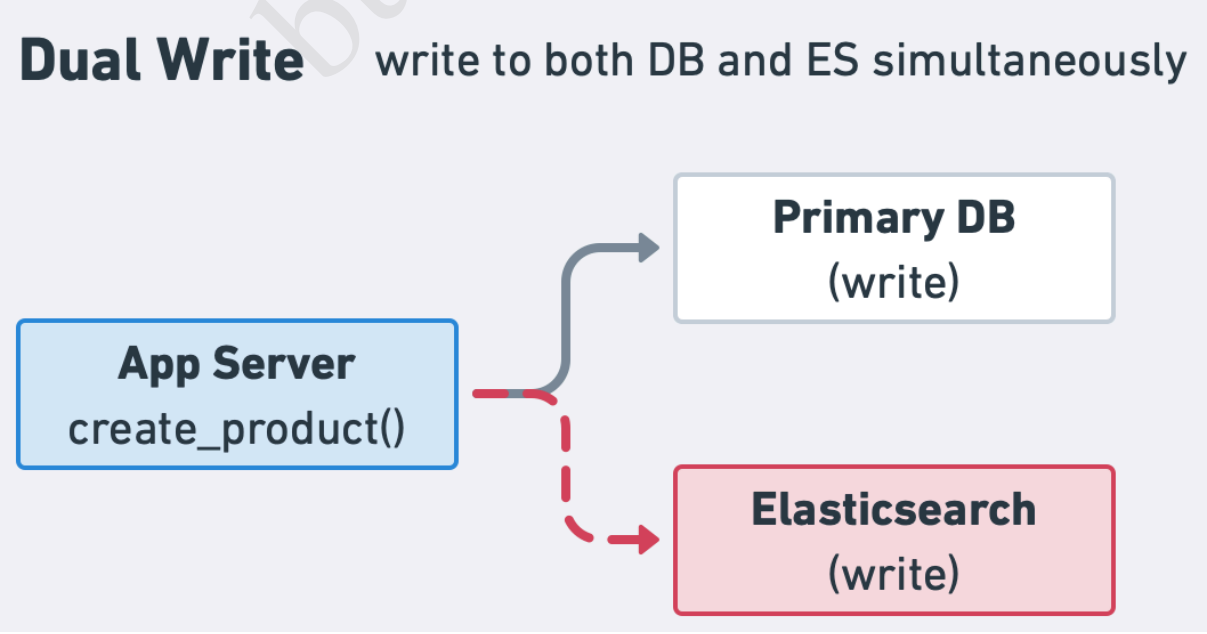
Indexing Pipeline：資料如何進入搜尋索引

這是搜尋系統設計裡最容易被忽略、但實際上最複雜的部分。

方案一：雙寫（Dual Write）

應用程式在寫入主資料庫的同時，也寫入搜尋索引。

```
def create_product(product_data):
    # 同時寫入兩個地方
    db.insert("products", product_data)
    elasticsearch.index("products", product_data)
```

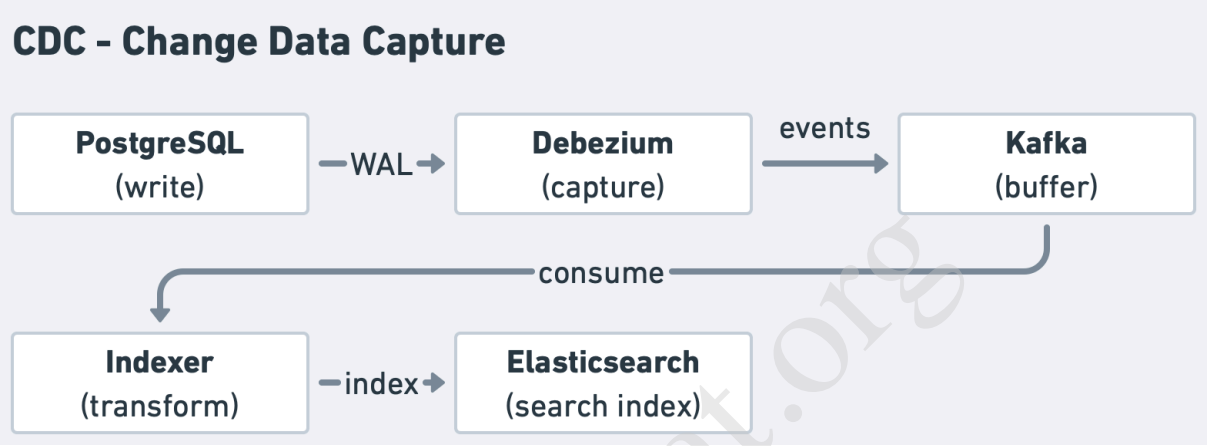


buildmoat.org

問題：兩個寫入不是原子的。如果資料庫成功但 Elasticsearch 失敗，資料就不一致了。如果 Elasticsearch 暫時不可用，怎麼補救？雙寫把複雜度推到了應用層，容易出錯。

方案二：CDC（Change Data Capture）

監聽資料庫的 WAL（Write-Ahead Log），把每一個資料變更轉換成事件，非同步同步到搜尋索引。



優點：

- 應用程式只需要寫主資料庫，搜尋同步完全解耦
- 中間有 Kafka 緩衝，Elasticsearch 暫時不可用也不會遺失資料
- 可以在 Indexer Service 裡做資料轉換（合併多個表的資料、重新計算欄位）

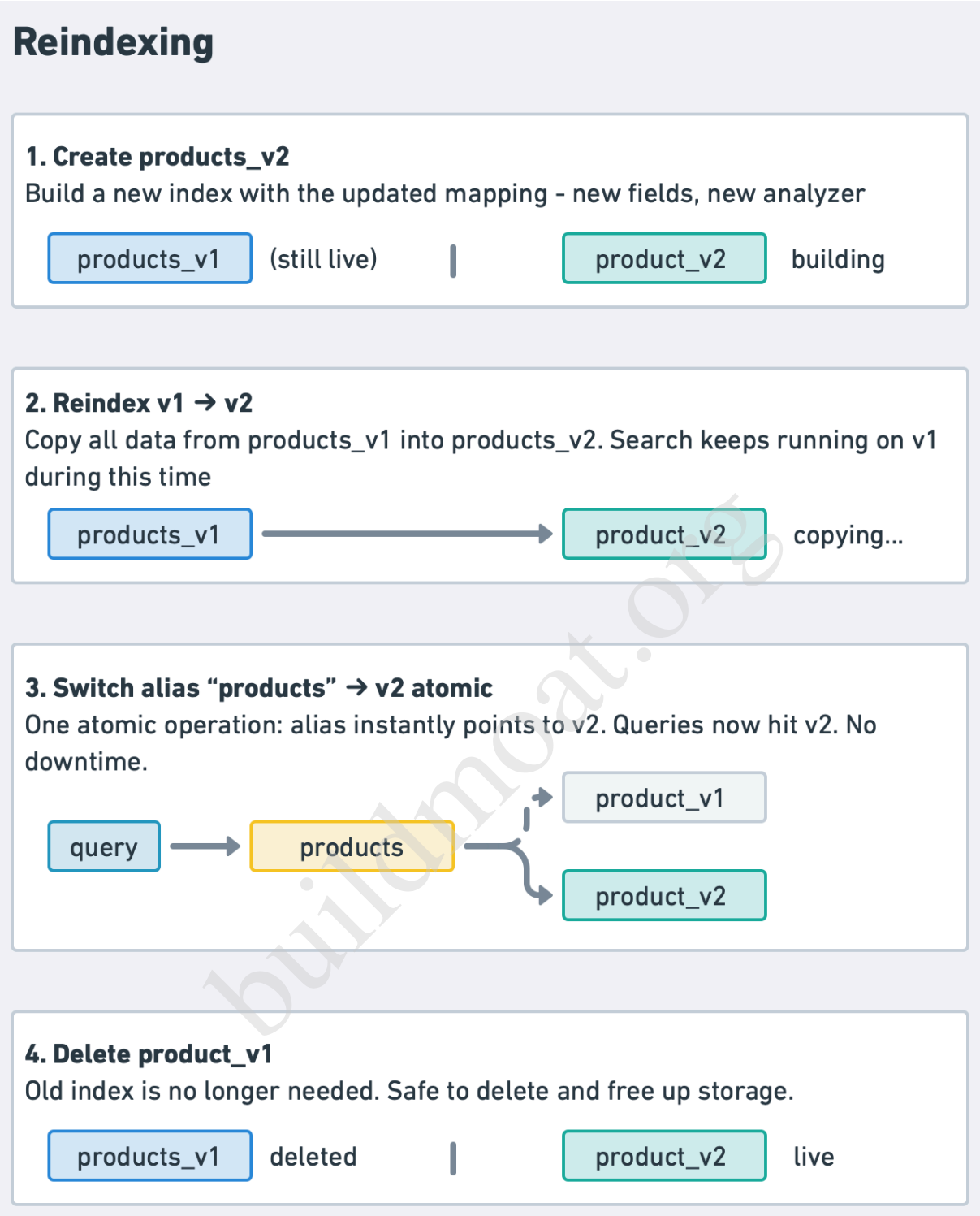
缺點：資料有延遲，從寫入到可搜尋通常是幾秒到幾十秒。

CDC 是最推薦的方案，大多數生產系統都走這條路。

重建索引（Reindexing）

當你改變了 mapping（例如新增欄位、換分詞器），需要對整個索引做重建。直接刪除重建會讓搜尋在重建期間失效，正確的做法是：

buildnoat.org



用 **alias** 做零停機的索引切換，是 Elasticsearch 的標準做法。

相關性排序（Relevance Ranking）

找到相關文件只是第一步，接下來要決定哪些排在前面。

BM25

Elasticsearch 的預設排序演算法。核心思想是：

- **TF（Term Frequency）**：這個詞在文件裡出現越多次，分數越高，但有上限，不是線性增長
- **IDF（Inverse Document Frequency）**：這個詞在所有文件裡越少見，分數越高。「鞋」出現在每個商品裡，它的 IDF 低；「Gore-Tex」只出現在少數商品，它的 IDF 高，更有辨別力
- **欄位長度**：同樣出現一次，在短標題裡比在長描述裡的權重更高

Boosting（欄位權重）

不是所有欄位的相關性都相同。商品名稱裡出現關鍵字，比商品描述裡出現更重要：

```
{
  "query": {
    "multi_match": {
      "query": "running shoes",
      "fields": ["name^3", "category^2", "description^1"]
    }
  }
}
```

`name^3` 表示名稱欄位的權重是描述欄位的 3 倍。

業務邏輯排序

純粹的文字相關性分數往往不夠。真實的搜尋排序通常是**相關性 + 業務指標**的組合：

```
最終分數 = 相關性分數 × 0.6 + 銷量分數 × 0.2 + 評分分數 × 0.1 +
新品加成 × 0.1
```

這讓你能夠在保持相關性的前提下，把高銷量、高評分的商品往前推。

常見功能的設計

Autocomplete（自動補全）

用戶輸入「runn」，系統即時返回「running shoes」、「running socks」、「runner backpack」等建議。

buildmoat.org

Edge N-gram：在 indexing 時，把「running」拆成 `["r", "ru", "run", "runn", "runni", "runnin", "running"]`，每個前綴都建索引。查詢時直接用前綴詞精確匹配，速度極快。

```
{
  "settings": {
    "analysis": {
      "tokenizer": {
        "autocomplete_tokenizer": {
          "type": "edge_ngram",
          "min_gram": 1,
          "max_gram": 20
        }
      }
    }
  }
}
```

補全詞的來源：

- **靜態**：把所有商品名稱預先建入補全索引，簡單但不反映熱度
- **動態**：從用戶的搜尋歷史統計高頻詞，熱門搜尋詞排在前面

延遲要求：Autocomplete 需要在用戶輸入每個字元後 100ms 內回應，否則體驗很差。通常在 Autocomplete 索引上加快取（Redis），或者用 Elasticsearch 的 `suggest` API，它針對補全場景做了特別優化。

Faceted Search（多面向過濾）

電商搜尋側欄的「品牌」、「價格範圍」、「顏色」過濾。點選之後，搜尋結果縮小，同時各個過濾選項的計數也跟著更新（例如「Nike (143)」、「Adidas (89)」）。

```
{
  "query": { "match": { "name": "running shoes" } },
  "aggs": {
    "brands": {
      "terms": { "field": "brand.keyword", "size": 10 }
    },
    "price_ranges": {
```

buildmoat.org

```
      "range": {
        "field": "price",
        "ranges": [
          { "to": 50 },
          { "from": 50, "to": 100 },
          { "from": 100 }
        ]
      }
    }
  }
}
```

Elasticsearch 的 Aggregation API 讓這件事變得直接：一次查詢同時返回結果和各個維度的統計數字。

分頁（Pagination）

Elasticsearch 提供三種分頁方式，實作細節見 elasticsearch。設計上的選擇原則：

- **From/Size**：有頁碼的搜尋（電商網站），但限制最大 offset（通常 10,000），避免深分頁的效能問題
- **Search After**：無限下拉的 feed（手機 App），效能穩定但不能跳頁
- **PIT Cursor**：需要在翻頁過程中保持資料一致視角時使用，成本最高

搜尋系統的擴展

Elasticsearch 的 Sharding 和 Replica

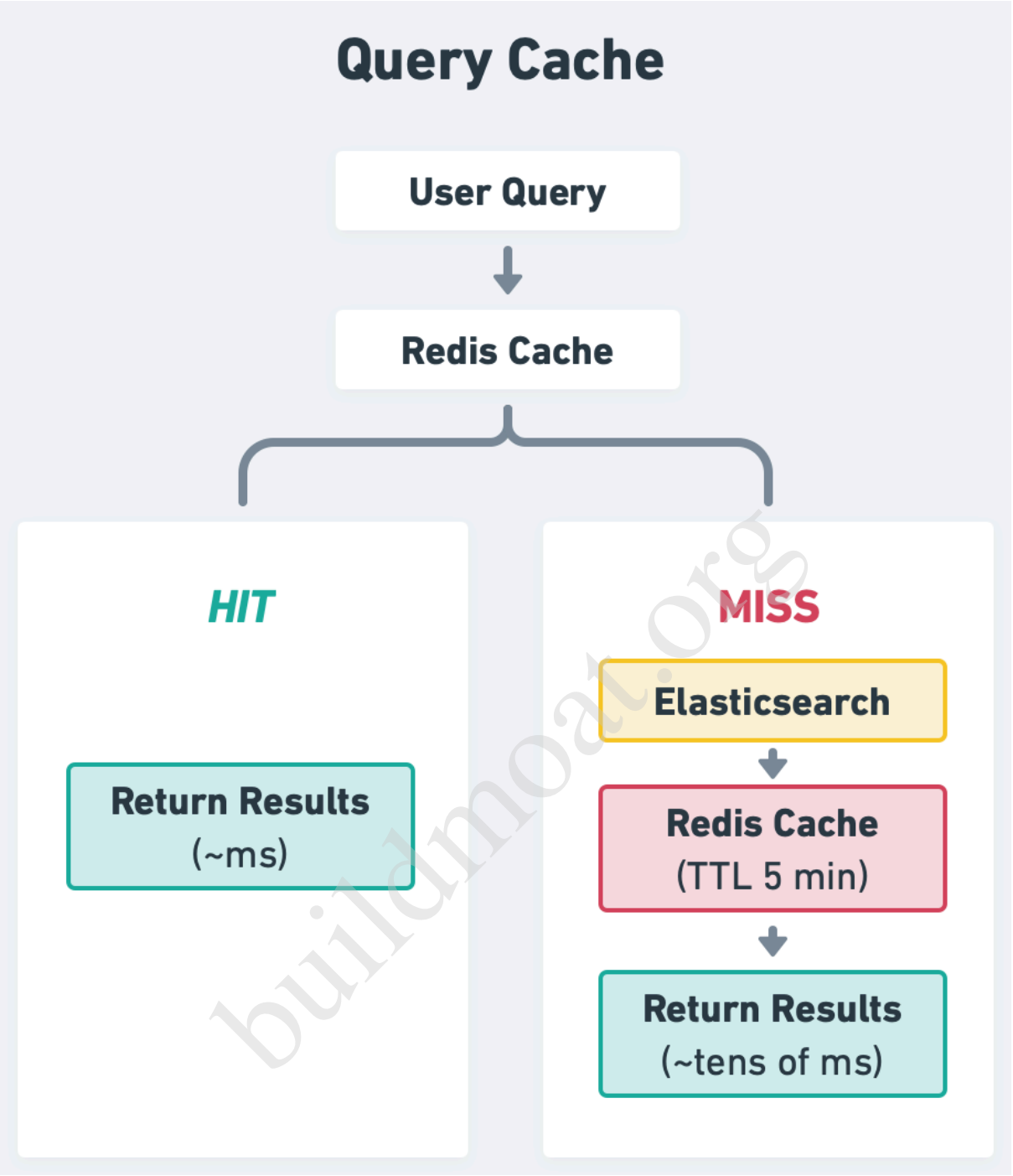
Elasticsearch 透過 Shard 把索引分散到多個節點，查詢並行執行後在 Coordinator 節點合併；Replica 提供讀取吞吐量和容錯。架構細節見 elasticsearch。

設計上最重要的決策是 **Shard 數量**，它在建立索引時就固定，之後無法修改（資料路由依賴 Shard 數量），只能靠 Reindex 解決。官方建議每個 Shard 控制在 10GB 到 50GB 之間，估算方式：預估資料量 ÷ 目標 Shard 大小。例如預估索引最終 500GB、目標每 Shard 25GB，就設 20 個 Primary Shard。寧可設多一點，避免之後被迫 Reindex。

查詢快取

熱門的搜尋詞（「Nike 跑步鞋」、「iPhone 手機殼」）會被大量重複查詢。在搜尋結果上加一層 Redis 快取，可以大幅降低 Elasticsearch 的負載：

buildmoat.org



buildmoat.org

快取的 key 是查詢詞 + 過濾條件的組合，TTL 設短一點（幾分鐘），讓新商品、下架商品能及時反映在搜尋結果裡。

什麼時候在面試裡用這些

任何有搜尋功能的系統

設計 Twitter、YouTube、Airbnb、LinkedIn……幾乎所有 consumer 產品都有搜尋。主動說明你不會用 `LIKE` 查詢，而是有獨立的搜尋索引：

「搜尋功能我會用 Elasticsearch 建立獨立的搜尋索引，資料透過 CDC + Kafka 從主資料庫非同步同步。這樣搜尋和業務系統完全解耦，搜尋索引的壓力不影響主資料庫。」

常見面試情境

設計 Twitter 搜尋：「推文寫入 MySQL 後，CDC 捕捉變更事件，發到 Kafka，Indexer 消費後寫入 Elasticsearch。搜尋時對推文內容、用戶名、hashtag 做多欄位搜尋，用發文時間和熱度（轉推數、按讚數）做 boosting，讓熱門和即時的推文排在前面。」

設計電商搜尋：「商品資料從 PostgreSQL 透過 CDC 同步到 Elasticsearch。搜尋結果用 BM25 做基礎相關性，再乘以銷量和評分的業務指標。側欄的品牌、價格、顏色過濾用 Aggregation 實現。Autocomplete 用 edge n-gram 索引，加 Redis 快取，確保 100ms 以內回應。」

設計 Airbnb 搜尋：「房源搜尋涉及地理位置過濾（在某個城市或某個距離範圍內），Elasticsearch 的 geo_distance 查詢可以處理這個。日期可用性的過濾比較複雜，像是每個房源都有一個可用日期的集合，需要設計成一個可以高效過濾的資料結構，通常是用 nested 或 join 的方式存在 Elasticsearch 裡。」

常見的 Deep Dive 問題

「搜尋索引和主資料庫的資料不一致怎麼辦？」

「在 CDC 方案裡，資料從主資料庫到搜尋索引有幾秒到幾十秒的延遲，這段時間內兩邊是不一致的。大多數搜尋場景可以接受這個延遲，比如用戶新發表的商品幾秒後才可以被搜到，是合理的。

如果不能接受，可以在寫入主資料庫的同時，用非同步任務直接觸發 Elasticsearch 的索引更新，延遲可以降到 1 秒以內，但這需要處理失敗重試和冪等性。

真正需要即時一致性的場景（例如庫存狀態），通常不放在搜尋索引裡，而是在從搜尋索引拿到候選結果後，再去主資料庫做最終的庫存驗證。」

「Shard 數量怎麼決定？」

「Elasticsearch 官方建議每個 Shard 的大小控制在 10GB 到 50GB 之間，單個 Shard 太大會讓查詢變慢，太小又浪費資源。

buildmoat.org

估算方式：預估一年內的資料量，除以目標 Shard 大小，得到 Shard 數量。例如預估索引最終 500GB，目標每個 Shard 25GB，就設 20 個 Primary Shard。寧可設多一點，因為 Shard 數量之後無法修改，少了就只能 Reindex。」

「搜尋結果的排序被人為操控（刷評分、刷銷量）怎麼辦？」

「業務指標排序確實有被操控的風險。幾個防範方向：用滾動時間窗口的銷量而不是累計銷量，讓歷史刷量逐漸失效；對異常的行為做偵測（短時間大量購買、評論內容雷同）；用多個維度的指標，不只依賴單一數字。完全消除操控很難，但可以讓操控的成本足夠高。」

總結

搜尋系統的設計圍繞兩條管線：

Indexing Pipeline（讓資料可以被搜尋）：

- 主資料庫負責寫入和一致性，搜尋索引是次要索引
- 用 CDC + Kafka 非同步同步，解耦且可靠
- 文字分析（斷詞、stemming）決定了什麼能被找到
- 用 Alias 實現零停機的 Reindex

Query Pipeline（讓搜尋結果有意義）：

- BM25 + Boosting 處理相關性排序
- 業務指標（銷量、評分）混入排序，而不是純粹文字相關性
- Autocomplete 用 Edge N-gram 索引，加 Redis 快取保住延遲
- Faceted Search 用 Aggregation 一次查詢返回結果和統計
- 分頁用 Search After 而不是大 offset

面試中，說明你的搜尋不走資料庫 `LIKE` 查詢，而是有獨立的搜尋索引和 indexing pipeline，當你說出這一句話就展示了你理解搜尋是一個獨立的設計問題。

bullmoat.org